

Plancast CLI - Product Requirements Document

Version: 1.0
Date: 2026-09-20
Author: PRD Generator
Status: Draft

1. Executive Summary

Plancast CLI converts verbose Markdown plans into concise, fact-faithful, two-host audio explanations that can be understood in either approximately two or five minutes. It is designed primarily for developers and technical users who receive long plans from AI coding agents and want to understand the proposed work without reading the entire file. A single terminal command will analyze the plan, create a structured `HOST_A` and `HOST_B` dialogue, synthesize two distinct voices, save an Apple-friendly M4A file, and optionally play it immediately. The expected outcome is faster plan comprehension without hiding decisions, risks, assumptions, or unresolved questions.

2. Problem Statement

Current state

AI coding agents frequently write long Markdown plans containing background, implementation details, alternatives, risks, and task sequences. These plans are useful as implementation records, but they are expensive to review when the user only needs a fast mental model before approving or redirecting the work. Existing text-to-speech tools generally read the source verbatim, while general podcast generators may introduce unsupported claims or omit decision-critical details.

Pain points

- 1. Users must scan an entire plan to identify its actual proposal, tradeoffs, and next actions.
- 2. Verbatim text-to-speech preserves the document's length and awkward Markdown structure.
- 3. Generic summaries may omit risks, assumptions, blockers, or unresolved decisions.
- 4. Manually prompting an AI, copying the dialogue, selecting voices, joining audio, and playing the result requires too many steps.
- 5. Cloud AI and speech services may receive plan contents without a sufficiently clear disclosure.

Impact

Slow plan review interrupts development flow and encourages users either to approve work without understanding it or to spend several minutes reconstructing the plan's meaning. A reliable two- or five-minute audio briefing should reduce review effort while retaining the information needed to make a decision.

3. Goals and Success Metrics

Goal	Metric	Target	Measurement Method
Reduce plan review effort	Median time from command start to user understanding	Under 3 minutes for 2-minute mode and under 6 minutes for 5-minute mode	Moderated test with 10 representative plans
Preserve decision-critical content	Plans for which proposal, major risk, and next action are correctly stated	At least 90%	Human comparison of source, script, and checklist
Produce predictable duration	Final audio duration	110-140 seconds for 2-minute mode; 270-330 seconds for 5-minute mode	Automated media-duration test
Make one-command use reliable	Successful runs for valid Markdown fixtures when providers are available	At least 95% across 100 fixture runs	Automated integration test
Keep repeated use fast and economical	Cache hits for unchanged input avoid provider calls	100%	Provider mock call-count test
Make cloud handling understandable	Users who correctly identify whether content leaves the Mac	At least 90% in a five-person usability check	Post-task question

Non-goals for version 1

- A full native macOS application.
- A hosted web service or user account system.
- Team libraries, sharing, feeds, subscriptions, or podcast publishing.
- Reading the Markdown source verbatim.
- Voice cloning or imitation of real people.
- Support for PDF, DOCX, webpages, or arbitrary folders.
- Guaranteed offline generation; local-provider adapters may be added later.

4. User Personas

AI-assisted developer

- **Role:** Software developer using Codex, Claude Code, or another coding agent.
- **Goals:** Understand a generated plan quickly before approving implementation.
- **Pain points:** Verbose plans, limited attention, and uncertainty about what the agent will actually change.
- **Technical proficiency:** High.
- **Usage context:** Runs Plancast from a repository immediately after an agent creates or updates a plan file.

Technical founder or product lead

- **Role:** Reviews implementation plans but may not need every code-level detail.
- **Goals:** Understand scope, customer impact, sequencing, risks, and decisions.
- **Pain points:** Technical documents bury the product consequence beneath implementation detail.
- **Technical proficiency:** Medium to high.
- **Usage context:** Listens while switching tasks, walking, or preparing feedback for the agent.

5. Product Principles

1. **Explain, do not recite:** Convert the plan into a spoken mental model rather than reading headings and bullets aloud.
2. **Faithfulness over entertainment:** Never invent project facts, decisions, or examples. Clearly label interpretation.
3. **Decision-first compression:** Preserve the proposal, rationale, risks, dependencies, open questions, and next action.
4. **One command:** A normal run must not require manual script editing, audio stitching, or opening another application.
5. **Transparent data handling:** State when plan text will be sent to external providers before the first provider request.
6. **Script as the canonical artifact:** Audio can be regenerated; the dialogue and metadata explain what was generated and how.

6. Functional Requirements

FR-001: Markdown input and command execution

Description: Accept one readable Markdown file and run the complete conversion workflow from a stable `plancast` command.

User story: As an AI-assisted developer, I want to pass a plan file to one command so that I do not need a multi-step audio workflow.

Acceptance criteria:

- ☐ `plancast PLAN.md` accepts absolute and relative paths.
- ☐ The command rejects missing paths, directories, non-Markdown inputs, unreadable files, and files larger than 2 MB before making provider requests.
- ☐ `--help` documents all supported options and exits with code 0 in under 500 ms.
- ☐ Invalid input exits non-zero and prints a specific remediation without a stack trace unless `--debug` is set.
- ☐ The command handles UTF-8 Markdown files containing headings, lists, code fences, tables, and links.

Priority: P0 (MVP)

Dependencies: None

FR-002: Fact-faithful plan condensation and two-host dialogue

Description: Extract the plan's decision-critical content and produce a natural conversation between a curious guide and a practical explainer.

User story: As a reviewer, I want a short conversation that explains what the plan proposes so that I can make an informed decision without reading every line.

Acceptance criteria:

- ☐ The generated script uses the exact stable labels `HOST_A:` and `HOST_B:` for every spoken turn.
- ☐ The script states the core problem, proposed approach, major implementation stages, material risks, open questions, and immediate next action when those elements exist in the source.
- ☐ Dates, quantities, named technologies, explicit exclusions, and uncertainty are preserved when material to the plan.
- ☐ The script does not present an unsupported claim as source fact; interpretations are introduced as interpretation.
- ☐ Host A keeps the listener oriented through short questions and recaps; Host B explains concretely without stage directions or exaggerated banter.
- ☐ The final dialogue ends with a compact recap covering what will happen, why it matters, what remains uncertain, and what the user should decide or do next.
- ☐ Raw provider output is schema-validated and an invalid response is retried no more than two times before failing with a recoverable error.

Priority: P0 (MVP)

Dependencies: FR-001

FR-003: Two-voice audio rendering and artifact output

Description: Render the validated dialogue with two distinct synthetic voices and create a playable M4A artifact.

User story: As a reviewer, I want a finished audio file so that I can listen instead of reading the generated dialogue.

Acceptance criteria:

- ☐ Host A and Host B use distinct configurable synthetic voices.
- ☐ Turns are rendered in source order with 200-500 ms of silence between speakers.
- ☐ The default output is `<source-basename>.plancast.m4a` in the source directory.
- ☐ `--output <path>` overrides the default and creates missing parent directories only when the parent of the requested output already has a writable ancestor.
- ☐ The final M4A can be opened by QuickTime Player and inspected by `afinfo`.
- ☐ Temporary per-turn audio is removed after successful assembly and retained under a reported debug directory after a failed assembly.
- ☐ Existing output files are not overwritten unless `--force` is supplied or the artifact is an exact cache hit.

Priority: P0 (MVP)

Dependencies: FR-002

FR-004: Two-minute and five-minute modes

Description: Offer explicit `2m` and `5m` briefing modes with different information depth and bounded audio duration.

User story: As a reviewer, I want to choose a quick or detailed explanation so that the briefing matches the importance of the plan.

Acceptance criteria:

- ☐ `--length 2m` targets 110-140 seconds and prioritizes the problem, proposal, major risk, decision, and next action.
- ☐ `--length 5m` targets 270-330 seconds and additionally covers architecture, phases, dependencies, alternatives, and unresolved questions present in the source.
- ☐ The default is `2m` and is shown in `--help` and the preflight summary.
- ☐ The generator computes a word budget from configured speaking speed and rejects scripts more than 15% outside that budget before TTS.
- ☐ A single automatic compression or expansion pass is allowed when the draft misses its word budget; subsequent failure stops before TTS.

Priority: P1 (Important)
Dependencies: FR-002, FR-003

FR-005: Immediate playback

Description: Optionally play the generated or cached audio after a successful run.

User story: As a terminal user, I want the audio to start immediately so that I do not need to locate and open the output file.

Acceptance criteria:

- ☐ `--play` starts the completed M4A with `/usr/bin/afplay` on macOS.
- ☐ Playback begins only after the final artifact passes duration and file-size validation.
- ☐ `Ctrl-C` stops playback and exits with code 130 without deleting the completed artifact.
- ☐ Generation success remains success if playback cannot start; the warning includes the output path and a manual playback command.

Priority: P1 (Important)
Dependencies: FR-003

FR-006: Content-addressed cache

Description: Reuse existing script and audio artifacts when the source and generation configuration have not changed.

User story: As a frequent user, I want unchanged plans to replay instantly so that I avoid unnecessary cost and waiting.

Acceptance criteria:

- ☐ The cache key includes normalized source bytes, length mode, prompt version, selected providers, models, voices, speaking speed, and output format.
- ☐ A valid cache hit makes zero LLM and TTS network requests.
- ☐ `--no-cache` bypasses cache reads and writes for the current run.
- ☐ `plancast cache clear` removes only Plancast cache entries after reporting the exact cache path and requesting confirmation, unless `--yes` is supplied.
- ☐ A corrupt or incomplete cache entry is ignored, logged as a warning, and replaced after a successful run.

Priority: P1 (Important)
Dependencies: FR-002, FR-003

FR-007: Configuration and cloud disclosure

Description: Configure providers and voices while clearly communicating when source text leaves the Mac.

User story: As a privacy-conscious user, I want to know which services receive my plan so that I can make an informed choice.

Acceptance criteria:

- ☐ Configuration precedence is command flags, environment variables, project config, user config, then documented defaults.
- ☐ Before the first uncached remote request, stderr states that plan content will leave the Mac and names the LLM and TTS providers.
- ☐ `--yes` can acknowledge the disclosure for non-interactive automation; without prior acknowledgement, an interactive terminal requires confirmation.
- ☐ A non-interactive run without cached artifacts and without `--yes` fails before transmitting content and explains how to continue.
- ☐ API keys are read from environment variables or macOS Keychain and are never written to config, metadata, logs, or errors.
- ☐ `--dry-run` validates input, resolves configuration, displays providers and output paths, and makes no provider requests or file writes.

Priority: P1 (Important)
Dependencies: FR-001

FR-008: Script and metadata sidecars

Description: Save inspectable artifacts that make the generated audio reproducible and auditable.

User story: As a technical reviewer, I want to inspect the exact dialogue and generation settings so that I can verify the briefing against the plan.

Acceptance criteria:

- ☐ A successful run saves `<basename>.plancast.txt` containing only the stable host dialogue.
- ☐ A successful run saves `<basename>.plancast.json` conforming to the `GenerationManifest` model in this PRD.
- ☐ Metadata records hashes, provider/model identifiers, voice identifiers, duration target, actual duration, prompt version, timestamps, and cache status.
- ☐ Metadata never contains API keys or full provider responses.
- ☐ Audio, dialogue, and manifest are written atomically so a failed run does not leave artifacts that appear complete.

Priority: P1 (Important)
Dependencies: FR-002, FR-003

FR-009: Finder Quick Action and native wrapper

Description: Provide a later macOS convenience layer that invokes the same CLI engine from Finder or a lightweight native interface.

User story: As a less terminal-oriented reviewer, I want to right-click a plan and create a briefing so that I can use Plancast without remembering commands.

Acceptance criteria:

- ☐ Finder exposes `Create 2-minute Plancast` for a selected `.md` file.
- ☐ The action invokes the installed CLI rather than duplicating generation logic.
- ☐ Success reveals the M4A in Finder and optionally starts playback.
- ☐ Errors show a concise message and a path to a detailed log.

Priority: P2 (Nice to have)
Dependencies: FR-001 through FR-008

Priority summary

Priority	Features	Share
P0	FR-001, FR-002, FR-003	33%
P1	FR-004, FR-005, FR-006, FR-007, FR-008	56%
P2	FR-009	11%

7. Non-Functional Requirements

Performance

- On an Apple silicon Mac with a broadband connection, a cache miss should complete within 45 seconds at p50 and 90 seconds at p95 for 2-minute mode, excluding provider outages.
- A cache hit should resolve and begin playback within 750 ms at p95.
- Input parsing and cache-key calculation should finish within 500 ms for files up to 2 MB.
- The process should use less than 300 MB resident memory for a 2 MB input.

Reliability

- Every successful exit must correspond to validated script, audio, and manifest artifacts.
- Network requests use a 30-second timeout and at most two retries with exponential backoff and jitter for transient `429` and `5xx` responses.
- Authentication, invalid input, unsafe overwrite, and schema-validation failures are never retried automatically.
- Writes use temporary files followed by atomic rename on the same filesystem.

Security and privacy

- No authentication or account system is required.
- Provider traffic uses HTTPS through official or standards-compliant SDKs.
- Secrets are accepted through environment variables or macOS Keychain only.
- Logs redact authorization headers, keys, and provider request bodies by default.
- Telemetry is disabled by default; version 1 collects no plan contents, filenames, scripts, audio, or usage events.
- Generated artifacts inherit the current user's default filesystem permissions and must not be made world-writable.

Compatibility

- Supported operating systems: macOS 14 Sonoma and later.
- Supported architectures: Apple silicon for launch; Intel is best-effort until CI coverage is added.
- Supported Node.js runtime: active LTS version selected at implementation time, with a minimum documented version and a startup compatibility check.
- Output format: AAC-LC audio in an M4A container, 44.1 kHz or 48 kHz, mono or stereo as determined by the renderer.

Accessibility

- The saved text dialogue provides a complete non-audio alternative.
- Terminal output does not rely on color alone and honors `NO_COLOR`.
- Progress output is suppressed or converted to stable line-oriented events when stdout is not a TTY.

8. Technical Architecture

System overview

Plancast is a local command-line orchestrator. It reads a Markdown file locally, normalizes it, computes a content-addressed cache key, and checks for a complete cached result. For a cache miss, it sends the source to a configured language-model adapter with a structured, versioned prompt and validates the returned dialogue. It sends each dialogue turn to a configured speech adapter, assembles the returned audio with macOS media tools, validates the duration, then atomically writes the M4A, dialogue, and manifest. When `--play` is present, the CLI launches `afplay` only after artifact validation succeeds.

Technology stack

- **CLI language:** TypeScript targeting Node.js.
- **Command framework:** `commander` for subcommands, flags, validation, and help output.
- **Schema validation:** `zod` for provider responses, configuration, and manifests.
- **Markdown parsing:** `unified` with `remark-parse` to extract structure while preserving code and table context.
- **LLM adapter:** OpenAI-compatible Responses or chat adapter behind a provider-neutral interface.
- **TTS adapter:** Existing Plancast OpenAI speech renderer logic, migrated behind a provider-neutral interface.
- **Audio assembly:** macOS `/usr/bin/afconvert` plus deterministic PCM/WAV concatenation; use `afinfo` for validation.
- **Playback:** macOS `/usr/bin/afplay`.
- **Testing:** Vitest for unit and integration tests; provider adapters use fixtures and local mocks.
- **Packaging:** npm package exposing the `plancast` binary, with an optional Homebrew formula after MVP validation.

Architecture flow

1. CLI parses flags and resolves the source file.
2. Input service validates size, encoding, extension, and readability.
3. Configuration service resolves flags, environment, project configuration, and user configuration.
4. Privacy preflight reports remote providers before any uncached external request.
5. Cache service calculates a SHA-256 key over source bytes and generation configuration.
6. Script service asks the LLM adapter for structured plan analysis and host turns.
7. Dialogue validator checks schema, required content categories, word budget, and safe speaker labels.
8. Speech service renders validated turns through the TTS adapter with bounded concurrency.
9. Audio assembler joins turns and pauses, converts to M4A, and verifies duration and file integrity.
10. Artifact writer atomically publishes the audio, dialogue, and manifest.

11. Playback service launches `afplay` when requested.

Provider boundaries

The core workflow must depend on `ScriptProvider` and `SpeechProvider` interfaces rather than provider SDK types. This keeps cloud disclosure accurate, makes provider behavior mockable, and leaves a clean path for local generation. Provider-specific request/response payloads must remain inside adapter modules.

9. Command and Internal Interface Specifications

No HTTP API is required for version 1. The public API is the command-line interface; internal provider interfaces are typed application contracts.

Command: generate a briefing

```
plancast <source.md> \  
  [--length 2m|5m] \  
  [--output <file.m4a>] \  
  [--play] \  
  [--force] \  
  [--no-cache] \  
  [--yes] \  
  [--dry-run] \  
  [--json] \  
  [--debug]
```

Example request:

```
plancast ./PLAN.md --length 2m --play --yes
```

Human-readable success response:

```
Created PLAN.plancast.m4a (02:08)  
Script: PLAN.plancast.txt  
Manifest: PLAN.plancast.json  
Cache: miss  
Playing with afplay...
```

Machine-readable success response for `--json` :

```
{  
  "status": "success",  
  "sourcePath": "/repo/PLAN.md",  
  "audioPath": "/repo/PLAN.plancast.m4a",  
  "scriptPath": "/repo/PLAN.plancast.txt",  
  "manifestPath": "/repo/PLAN.plancast.json",  
  "targetLength": "2m",  
  "actualDurationSeconds": 128.4,  
  "cacheStatus": "miss",  
  "played": true  
}
```

Exit codes:

Code	Meaning
0	Generation or valid cache retrieval succeeded
1	Unexpected internal error

Code	Meaning
2	Invalid CLI arguments or input file
3	Configuration or missing credential error
4	Provider request or provider response error
5	Audio assembly or validation error
6	Unsafe output overwrite prevented
130	Interrupted by user

Command: inspect resolved work without executing

```
plancast ./PLAN.md --length 5m --dry-run --json
```

Response:

```
{
  "status": "dry_run",
  "sourcePath": "/repo/PLAN.md",
  "targetLength": "5m",
  "scriptProvider": "openai",
  "speechProvider": "openai",
  "contentLeavesMac": true,
  "wouldUseCache": true,
  "outputPath": "/repo/PLAN.plancast.m4a"
}
```

Internal interface: ScriptProvider

```
interface ScriptProvider {
  readonly id: string;
  readonly model: string;
  generateDialogue(request: DialogueRequest): Promise<DialogueResult>;
}

interface DialogueRequest {
  sourceMarkdown: string;
  targetLength: "2m" | "5m";
  wordBudget: { min: number; max: number };
  promptVersion: string;
}

interface DialogueResult {
  summary: PlanSummary;
  turns: DialogueTurn[];
  sourceGroundingNotes: string[];
}
```

Internal interface: SpeechProvider

```
interface SpeechProvider {
  readonly id: string;
  readonly model: string;
  synthesize(request: SpeechRequest): Promise<RenderedTurn>;
}

interface SpeechRequest {
  turnId: string;
  text: string;
  voice: string;
  speed: number;
  outputFormat: "wav";
}

interface RenderedTurn {
  turnId: string;
  audioPath: string;
  durationSeconds: number;
  sha256: string;
}
```

Provider error contract

```
{
  "code": "PROVIDER_RATE_LIMITED",
  "message": "The speech provider rate limit was reached after two retries.",
  "provider": "openai",
  "retryable": true,
  "requestId": "redacted-or-provider-safe-id"
}
```

10. CLI UX Requirements

Default generation experience

Purpose: Convert one plan with minimal interaction.

Key elements:

- Preflight line: source, selected length, providers, destination, and whether content leaves the Mac.
- Stable progress phases: Analyzing , Writing dialogue , Rendering voices , Assembling audio , and Validating .
- Completion summary: duration, cache status, and absolute artifact paths.

User flow:

1. User runs `plancast PLAN.md --play` .
2. CLI validates the file and configuration.
3. On the first uncached remote run, CLI displays the cloud disclosure and requests acknowledgement.
4. CLI generates and validates the dialogue, renders audio, and writes artifacts.
5. CLI plays the completed audio and leaves reusable sidecars beside it.

States:

- **Cached:** Show `Cache hit` , skip provider progress, and play or return the existing artifact.
- **Loading:** Show the current stable phase and elapsed time without fake percentage estimates.
- **Recoverable error:** Name the failed phase, preserve safe debug artifacts, and provide one concrete next command.
- **Non-interactive:** Emit stable line-oriented output or JSON; never wait for confirmation.

Help experience

Purpose: Make the command self-discoverable.

Key elements:

- One-sentence explanation.
- Three examples: default two-minute generation, five-minute generation, and immediate playback.
- Privacy note explaining cloud providers.
- Exit-code reference and environment variables.

Error-writing rules

- Lead with the actionable failure, not the exception class.
- Include the relevant file or provider, but never secrets or full source content.
- Give exactly one primary remediation and optionally one debug command.
- Send human progress and errors to stderr when `--json` reserves stdout for the result object.

11. Data Models

PlanSummary

Field	Type	Required	Description
thesis	string	Yes	One-sentence statement of what the plan proposes
problem	string	Yes	Problem or motivation stated by the source
approach	string[]	Yes	Ordered major stages or decisions
risks	string[]	Yes	Material risks explicitly present or clearly labeled interpretations
openQuestions	string[]	Yes	Unresolved decisions or unknowns
nextActions	string[]	Yes	Immediate actions expected from user or agent
sourceFacts	SourceFact[]	Yes	Material names, dates, numbers, and constraints used in the dialogue

Relationships: One `PlanSummary` belongs to one `GenerationManifest` and grounds many `DialogueTurn` records.

SourceFact

Field	Type	Required	Description
id	string	Yes	Stable identifier within the generation
claim	string	Yes	Concise fact extracted from the source
sourceHeading	string or null	Yes	Nearest Markdown heading when available
lineStart	integer or null	Yes	One-based source line for auditability
lineEnd	integer or null	Yes	One-based inclusive source line
qualifier	string or null	Yes	Uncertainty, condition, or scope qualifier

Relationships: Many `SourceFact` records belong to one `PlanSummary` ; many may ground each `DialogueTurn` .

DialogueTurn

Field	Type	Required	Description
id	string	Yes	Sequential stable identifier such as <code>turn-001</code>
host	enum HOST_A or HOST_B	Yes	Speaker label
text	string	Yes	Spoken text without stage directions
sourceFactIds	string[]	Yes	Facts grounding this turn
interpretation	boolean	Yes	Whether the turn contains labeled interpretation
estimatedWords	integer	Yes	Word count used for duration control

Relationships: Many turns belong to one `GenerationManifest` and reference zero or more `SourceFact` records.

GenerationManifest

Field	Type	Required	Description
schemaVersion	string	Yes	Manifest contract version
generationId	UUID	Yes	Unique generation identifier
createdAt	ISO 8601 timestamp	Yes	Completion time in UTC
sourcePath	string	Yes	Resolved source path at generation time
sourceSha256	string	Yes	Hash of exact source bytes
cacheKey	string	Yes	Hash of source plus generation configuration
promptVersion	string	Yes	Version of condensation prompt
targetLength	enum 2m or 5m	Yes	Requested mode
actualDurationSeconds	number	Yes	Validated final duration
scriptProvider	ProviderDescriptor	Yes	LLM provider and model
speechProvider	ProviderDescriptor	Yes	TTS provider and model
voiceA	string	Yes	Voice identifier for Host A
voiceB	string	Yes	Voice identifier for Host B
speed	number	Yes	Synthesis speaking-rate multiplier
cacheStatus	enum hit, miss, or bypass	Yes	Result of cache lookup
artifacts	ArtifactDescriptor[]	Yes	Paths, sizes, and hashes of final files

Relationships: One manifest owns one summary, many dialogue turns, and three or more artifact descriptors.

ProviderDescriptor

Field	Type	Required	Description
id	string	Yes	Stable provider identifier
model	string	Yes	Provider model identifier
remote	boolean	Yes	Whether content leaves the Mac

ArtifactDescriptor

Field	Type	Required	Description
kind	enum audio, script, or manifest	Yes	Artifact category
path	string	Yes	Absolute path at generation time
bytes	integer	Yes	File size
sha256	string	Yes	File content hash; for the manifest, hash is computed over its canonical payload excluding this descriptor

Configuration

Field	Type	Required	Description
scriptProvider	string	Yes	Registered provider adapter
scriptModel	string	Yes	Model used for dialogue generation
speechProvider	string	Yes	Registered TTS adapter
speechModel	string	Yes	Model used for speech
voiceA	string	Yes	Host A voice
voiceB	string	Yes	Host B voice
speed	number	Yes	Range 0.75-1.25
cacheDirectory	string	Yes	User cache path
remoteDisclosureAccepted	boolean	Yes	User-level acknowledgement, without source content

12. Integration Points

Script-generation provider

Purpose: Convert source Markdown into a structured summary and dialogue.

Integration type: HTTPS API through a provider adapter.

Data exchanged: Outbound source Markdown, length target, and generation instructions; inbound structured summary and host turns.

Authentication: Provider API key from environment or macOS Keychain.

Rate limits: Provider-specific; adapter normalizes 429 into a retryable application error.

Fallback behavior: Retry transient failures at most twice, then stop before TTS. Never silently switch providers because that changes data handling and output behavior.

Speech provider

Purpose: Render each validated host turn using one of two synthetic voices.

Integration type: HTTPS API through a provider adapter.

Data exchanged: Outbound dialogue text, voice, speed, and output format; inbound WAV audio bytes.

Authentication: Provider API key from environment or macOS Keychain.

Rate limits: Provider-specific; render with configurable concurrency defaulting to two requests.

Fallback behavior: Retry an individual turn at most twice for transient errors. Resume completed turns within the current generation, but do not publish partial final artifacts.

macOS audio tools

Purpose: Convert, inspect, and play generated audio.

Integration type: Local subprocess using absolute system paths.

Data exchanged: Local WAV/M4A paths and process exit status only.

Authentication: None.

Fallback behavior: If `afconvert` or `afinfo` is unavailable, fail with a compatibility error. If `afplay` is unavailable, keep the valid artifact and warn that automatic playback was skipped.

AI coding agents

Purpose: Allow agents to invoke Plancast after creating a plan.

Integration type: Shell command.

Data exchanged: Local file path and stable JSON result when requested.

Authentication: None beyond provider configuration.

Fallback behavior: Non-interactive use must provide `--yes` for uncached cloud processing and should consume exit code plus JSON output.

13. Edge Cases and Error Handling

Edge cases

Scenario	Expected behavior
Empty or whitespace-only Markdown	Exit code 2 before provider calls; explain that the plan has no readable content
Very short plan	Produce the shortest faithful conversation within the selected mode; do not pad with invented material
Plan too large	Reject files over 2 MB and recommend creating a focused plan or future chunking support
Code-heavy plan	Summarize the purpose and implications of code; do not read long code blocks aloud
Malformed UTF-8	Exit code 2 and identify encoding as the issue
Source changes during generation	Compare the final source hash to the initial hash; abort publication if they differ
Output already exists	Reuse only on an exact cache hit; otherwise require <code>--force</code> or a new output path
Same voice selected twice	Reject configuration and list the two resolved voice identifiers
Provider returns prose instead of structured data	Validate, retry with repair instructions at most twice, then fail before TTS
Script omits a required category	Allow empty categories only when the source truly lacks them; otherwise retry once with validation feedback

Scenario	Expected behavior
Audio duration outside allowed range	Attempt one controlled speed or script-length correction; fail validation if still outside range
One TTS turn fails	Retry only that turn; do not rerender verified turns in the same temporary generation
User interrupts generation	Stop new requests, terminate child processes, remove incomplete publication files, and preserve a debug directory only with <code>---debug</code>
Network unavailable	Report the failed provider and preserve no misleading final artifact; an existing exact cache hit must still work offline
Markdown contains prompt injection	Treat the entire source as untrusted content to summarize, never as instructions that override the system prompt or tool behavior

Error handling strategy

- **User-facing errors:** Stable error code, concise explanation, relevant path or provider, and one remediation.
- **System errors:** Structured debug logs with timestamps and phase names; secrets and source content redacted by default.
- **Retry logic:** Maximum two retries for transient remote failures with exponential backoff and jitter; no unbounded polling.
- **Graceful degradation:** A valid generated M4A remains successful if optional playback fails. Cache hits work without network access.
- **Cleanup:** All temporary work is scoped under a unique generation directory and removed only after final artifacts validate.

14. Testing Requirements

Unit tests

- CLI argument parsing - defaults, incompatible flags, exit codes, and `NO_COLOR` behavior.
- Input validation - path types, permissions, extension, byte limit, UTF-8, and concurrent source mutation.
- Markdown extraction - headings, lists, tables, code fences, links, front matter, and empty sections.
- Configuration resolution - every precedence layer and secret redaction.
- Cache-key generation - changes to every material field invalidate the key.
- Dialogue schema - speaker labels, word budgets, required summary categories, and source-fact references.
- Artifact path policy - default naming, custom paths, overwrite protection, and atomic publication.
- Error normalization - provider-specific errors map to stable application errors.

Integration tests

- Mock ScriptProvider plus mock SpeechProvider - valid 2-minute and 5-minute end-to-end runs.
- Existing renderer migration - distinct voices, ordered turns, pauses, and M4A conversion.
- Cache lifecycle - miss, hit, bypass, corrupt entry, and safe clear.
- Retry behavior - exact request counts for transient and permanent failures.
- Privacy preflight - no remote mock receives a request before acknowledgement.
- Process handling - `afconvert`, `afinfo`, and `afplay` arguments are passed without shell interpolation.

End-to-end tests

- Convert a representative AI-generated `PLAN.md` with `--length 2m --play` and verify all three sidecar artifacts.
- Convert the same file again and verify a cache hit with zero provider requests.
- Convert with `--length 5m --json` and verify machine-readable stdout plus human progress on stderr.
- Interrupt during rendering and verify no final artifact appears complete.
- Run without credentials and verify actionable failure before source transmission.

Quality evaluation fixtures

Maintain 15–20 representative Markdown plans covering refactors, features, migrations, bug investigations, releases, product proposals, and infrastructure changes. For every prompt-version change, score:

- Proposal accuracy.
- Risk preservation.
- Next-action accuracy.
- Unsupported-claim count.
- Duration compliance.
- Listening clarity.

Release requires zero material unsupported claims across the fixture set and at least 90% preservation of proposal, major risk, and next action.

Performance tests

- 2 MB Markdown parsing and hashing completes under 500 ms on the oldest supported Apple silicon test machine.
- Exact cache hit returns the result under 750 ms at p95 over 100 iterations.
- Mocked two-minute end-to-end orchestration excluding provider latency completes under 3 seconds.
- Memory remains below 300 MB for the maximum supported input.

15. Implementation Notes for AI

Assumptions

1. The repository is currently greenfield.
2. The existing Plancast plugin renderer under `/Users/setu/plugins/plancast` may be used as behavioral reference, but code should be imported deliberately with tests rather than edited in place.
3. Version 1 is macOS-first and terminal-first.
4. Initial script and speech adapters may use OpenAI-compatible services, but core modules must not depend directly on one provider.
5. Users provide and pay for their own provider credentials.
6. A native SwiftUI or Finder interface is explicitly deferred until the CLI workflow is validated.

Build order

1. Initialize TypeScript, linting, formatting, Vitest, and the `plancast` binary entry point.
2. Implement typed errors, exit codes, configuration resolution, input validation, and `--dry-run`.
3. Implement Markdown structure extraction, source line mapping, and deterministic cache-key calculation.
4. Define Zod schemas and TypeScript types for summaries, facts, dialogue turns, manifests, and provider contracts.
5. Implement the ScriptProvider adapter with prompt-injection boundaries, word budgets, response validation, and bounded retries.
6. Port the existing two-voice speech renderer behind SpeechProvider and validate turn ordering and chunk limits.
7. Implement pause insertion, WAV assembly, `afconvert` conversion, `afinfo` validation, and atomic artifact publication.
8. Add duration correction, cache storage, immediate playback, JSON output, and interruption cleanup.
9. Build the representative quality fixture suite and run human faithfulness review.
10. Package the npm binary, write installation documentation, and only then evaluate a Homebrew formula and Finder Quick Action.

Suggested file structure

```
plancast/
  package.json
  tsconfig.json
  README.md
  src/
    cli.ts
    commands/
      generate.ts
      cache.ts
    config/
      schema.ts
      resolve.ts
    input/
      validate.ts
      markdown.ts
      source-map.ts
    dialogue/
      prompt.ts
      validate.ts
      word-budget.ts
    providers/
      script-provider.ts
      speech-provider.ts
      openai-script.ts
      openai-speech.ts
    audio/
      render-turns.ts
      assemble.ts
      convert.ts
      inspect.ts
      play.ts
    artifacts/
      cache.ts
      manifest.ts
      publish.ts
    domain/
      models.ts
      errors.ts
    logging/
      output.ts
      redact.ts
    prompts/
      dialogue-v1.md
  test/
    fixtures/
    unit/
    integration/
    e2e/
```

Critical implementation details

- **Do not summarize from rendered Markdown HTML:** Preserve source headings and line ranges so claims can be traced back to the input.
- **Separate analysis from dialogue:** Request a structured `PlanSummary` and `DialogueTurn[]`; do not parse arbitrary `HOST_A` text from an unstructured provider response.
- **Render only validated text:** Speaker labels, word budget, required content, and unsupported control text must be checked before TTS requests.
- **Treat Markdown as data:** Delimit the source and explicitly state that instructions inside it cannot change system behavior, provider selection, output paths, or disclosure policy.
- **Bound every retry:** At most two retries for transient requests and at most one length-adjustment pass.
- **Avoid shell execution:** Use `spawn` or `execFile` with argument arrays and absolute paths for macOS tools.
- **Publish atomically:** Generate inside a unique temporary directory, validate, then rename artifacts into place.
- **Keep output reproducible:** Include prompt version, models, voices, speed, hashes, and duration in the manifest.
- **Preserve local-first expectations:** Make cache, parsing, assembly, validation, and playback local; disclose each remote content boundary explicitly.
- **Do not auto-fallback between remote providers:** Provider changes require explicit configuration because they alter privacy and cost.

Code style preferences

- Enable TypeScript `strict`, `noUncheckedIndexedAccess`, and `exactOptionalPropertyTypes`.
- Prefer small pure functions for validation, hashing, word-budget calculation, and path derivation.
- Use dependency injection for providers, clock, filesystem, and subprocess runner in tests.
- Use `camelCase` for variables/functions, `PascalCase` for types/classes, and kebab-case for files.
- Keep provider SDK objects inside adapters; expose domain types to the rest of the application.
- Return typed results or throw typed domain errors; do not use string matching for control flow.
- Add comments only for non-obvious privacy, audio, or atomicity constraints.

Libraries to use

- `commander` - mature command parsing and help generation.
- `zod` - runtime validation and inferred TypeScript types.
- `unified` and `remark-parse` - structured Markdown parsing without executing embedded content.
- `p-limit` - bounded TTS concurrency.
- `tempy` or a small internal helper using `fs.mkdtemp` - safely scoped temporary directories.
- `vitest` - fast unit and integration tests with TypeScript support.
- Official provider SDKs only inside adapters - authentication and transport maintenance.

Libraries and approaches to avoid

- Electron - unnecessary footprint for a terminal-first product.
- A database - content-addressed files and manifests are sufficient for version 1.
- Shell command strings - introduce quoting and injection risk.
- Generic agent frameworks - add abstraction without helping the bounded source-to-script pipeline.
- Automatic voice cloning - outside product scope and creates consent risk.
- Telemetry SDKs - conflict with the local, private default and are not needed to validate MVP usefulness.

Common pitfalls

- **Audio that is short but unhelpful:** Enforce required content categories before optimizing conversational style.
- **A summary that sounds certain when the plan is tentative:** Preserve qualifiers and explicitly identify unresolved questions.
- **Duration controlled only by TTS speed:** Use a word budget first; excessive speed damages comprehension.
- **Provider output treated as trusted:** Validate every response and never allow it to choose paths or commands.
- **Cache poisoning or stale reuse:** Include all material configuration and prompt versions in the cache key and validate artifact hashes.
- **Misleading success after partial failure:** Success requires complete validated artifacts; playback is the only optional post-success step.
- **Leaking source text through logs:** Log phase, provider, timing, and safe IDs rather than prompts or provider bodies.
- **Overbuilding the native interface:** Validate the command and audio quality before starting SwiftUI work.

Testing approach

- Write unit tests alongside each domain module before provider integration.
- Use recorded synthetic fixtures only when their licensing and redaction are clear; prefer local provider mocks for CI.
- Keep one opt-in live integration suite requiring provider credentials and never run it by default.
- Snapshot structured manifests and dialogue models, not unstable human progress formatting.
- Gate releases on automated tests, M4A inspection, duration limits, and human review of the representative plan fixture set.

16. Release Plan

Phase 0 - Feasibility

- Port the existing renderer into a standalone experimental command.
- Generate 2-minute and 5-minute samples from five representative plans.
- Confirm duration, two-voice distinction, M4A compatibility, cost, and factual faithfulness.

Exit criterion: At least four of five briefings preserve the proposal, risk, and next action with no material unsupported claim.

Phase 1 - MVP

- Deliver FR-001 through FR-003 with explicit provider configuration and manual privacy acknowledgement.
- Add typed manifests, fixture tests, atomic output, and documented installation.

Exit criterion: A new user can install the command and create a valid M4A from `PLAN.md` in one documented command.

Phase 2 - Useful daily tool

- Deliver FR-004 through FR-008.
- Validate quality on 15-20 representative plans.
- Add npm packaging and evaluate Homebrew distribution.

Exit criterion: At least 90% decision-critical preservation, 95% successful fixture runs, and duration targets met across the quality set.

Phase 3 - Convenience surfaces

- Deliver FR-009 only after repeated CLI use demonstrates demand.
- Reuse the CLI engine from a Finder Quick Action or minimal native wrapper.

Exit criterion: The convenience surface introduces no separate generation logic and successfully handles the same fixtures as the CLI.

17. Open Questions

1. Which script-generation and speech models provide acceptable quality and cost for the initial default configuration?
2. Should user-level cloud acknowledgement persist indefinitely, per provider pair, or require renewal after configuration changes?
3. Should the default artifact location remain beside the source or move to a dedicated output directory for cleaner repositories?
4. Is AAC-LC M4A sufficient, or should the CLI also expose WAV for editing and debugging?
5. What speaking speed and voice pairing produce the best comprehension without making five-minute mode feel slow?
6. Should project-level configuration be committed as `.plancast.json`, or should all configuration remain user-local until team use is validated?

18. Definition of Done

- ☐ All P0 requirements and their acceptance criteria pass.
- ☐ Automated unit, integration, and end-to-end tests pass on the oldest supported macOS version in CI or a documented test host.
- ☐ Five representative plans pass manual source-to-script faithfulness review with no material unsupported claims.
- ☐ Both length modes produce valid M4A files within their required duration ranges.
- ☐ A repeated unchanged run is an exact cache hit with zero provider calls once caching is included.
- ☐ Cloud disclosure is shown before the first uncached remote request.
- ☐ No secrets or source contents appear in default logs or manifests beyond the user-requested script artifact.
- ☐ Installation, configuration, examples, privacy boundaries, error remediation, and uninstall instructions are documented.
- ☐ The native macOS application remains outside MVP scope unless new user evidence changes the product decision.